

ASTGuard

Adaptive Relation-Gated Structural Attention for C/C++ Vulnerability Detection

In-Depth Research Proposal

ECE 592 / CSC 591

September 2026

Core idea. Rather than treating program structure as a fixed mask or a globally weighted feature, ASTGuard learns, for each transformer layer, attention head, token, and structural relation, how strongly AST and data-flow information should influence self-attention. Normal semantic attention remains unrestricted.

Abstract

Software vulnerability detection is a difficult code-understanding problem because the security status of a function often depends on relationships between definitions, uses, control constructs, and operations rather than on isolated lexical cues. Pretrained code transformers such as CodeBERT provide strong token-level representations, while structure-aware models such as GraphCodeBERT and graph neural networks explicitly encode program relations. However, fixed structural mechanisms cannot express a basic possibility: structural information may be useful for some tokens, heads, and layers but unnecessary or even distracting for others. This proposal introduces ASTGuard, an adaptive relation-gated attention mechanism for function-level C/C++ vulnerability detection. ASTGuard augments selected self-attention layers with soft, relation-specific biases derived from Abstract Syntax Tree (AST) and Data Flow Graph (DFG) edges. A contextual gate determines the contribution of each structural relation at each token, head, and layer. The primary evaluation uses PrimeVul because its de-duplication, chronological split, severe class imbalance, and vulnerable/patched pairs create a more realistic test than older randomly split benchmarks. DiverseVul is used for cross-dataset generalization, and Devign is retained as a supplementary legacy benchmark when label semantics allow a fair comparison. The study is designed around controlled same-backbone ablations and strong external baselines, including CodeBERT, GraphCodeBERT, LineVul, ReGVD, and Devign. The central question is not simply whether AST/DFG features help, but whether an adaptive mechanism can learn when and where explicit program structure should influence a pretrained code model while reducing false alarms and preserving recall.

1. Problem, Motivation, and Scope

The task is function-level binary vulnerability classification for C and C++ source code. Given a function, the model predicts whether the function is vulnerable or non-vulnerable. The practical objective is not merely high overall accuracy. In realistic code bases, vulnerable functions are rare, so a model that raises many false alarms can be unusable even when accuracy appears strong. Accordingly, ASTGuard emphasizes precision, false-positive rate (FPR), precision-recall behavior, and PrimeVul's vulnerability-detection-oriented evaluation in addition to conventional F1 and recall.

The core motivation is structural. A token-sequence model can learn lexical and contextual patterns, but security flaws frequently depend on relationships that are distributed across a function: where a value is defined, where it flows, what operation consumes it, how a condition scopes subsequent statements, or whether a sanitization/validation step dominates a use. ASTs expose syntax and local structural organization; DFGs expose value dependencies and definition-use relations. The challenge is to incorporate these signals without forcing the model to follow them everywhere.

ASTGuard therefore keeps a pretrained transformer as the main representation learner and treats program structure as an additive, learnable prior. The project is intentionally scoped to function-level detection, C/C++, public datasets, and open-source pretrained models. It does not require proprietary LLM APIs, synthetic vulnerability generation, or remote inference services.

2. Research Gap and Novelty Positioning

The novelty claim must be precise. ASTGuard is not novel merely because it combines source tokens with AST or DFG information. Prior work already establishes several relevant directions. CodeBERT is a strong sequence-based pretrained transformer for code [1]. GraphCodeBERT explicitly incorporates data-flow structure and uses graph-guided masked attention during pretraining [2]. Devign and ReGVD represent vulnerability detection with graph neural networks or graph-based representations [5,6]. LineVul shows that a CodeBERT-based transformer can be highly competitive for vulnerability prediction and line-level localization [7]. PrimeVul further demonstrates that many earlier benchmark results can be misleading under realistic de-duplication, chronological splitting, imbalance, and vulnerable/patched evaluation [3].

ASTGuard instead studies a narrower mechanism: adaptive structural influence inside the self-attention of a pretrained code transformer. The proposed model preserves ordinary token-to-token attention and adds soft biases for structural relations. Those biases are modulated by gates conditioned on the current token representation and can differ by layer

and head. Thus, AST and DFG information are available but not mandatory. This separates ASTGuard from hard structural masks and from architectures that first produce a separate graph embedding and then fuse it with a sequence representation.

The defensible research claim is therefore methodological and empirical: determine whether token-conditioned, head- and layer-specific structural gating offers a better precision/false-positive/generalization tradeoff than (i) the identical sequence-only backbone, (ii) fixed structural biases using the same backbone, and (iii) established external structure-aware or vulnerability-specific baselines. The project should not claim to be the first-ever adaptive structure-aware transformer unless a systematic literature review confirms that stronger statement.

2.1 Proposed contributions

- Adaptive relation-gated structural attention that injects AST and DFG relations as soft additive attention priors rather than hard masks.
- A controlled causal-style ablation suite using the same CodeBERT backbone: sequence-only, AST-fixed, DFG-fixed, AST+DFG-fixed, and adaptive ASTGuard.
- Gate-level analysis that measures where AST and DFG relations are used across layers, heads, labels, CWEs, and correct/incorrect predictions.
- Evaluation under realistic imbalance, chronological/de-duplicated data, vulnerable/patched pairs, low-data settings, and cross-dataset transfer.
- A reproducible, no-proprietary-API pipeline that downloads public data/models, extracts structure locally, trains, evaluates, and produces paper-ready artifacts.

3. Research Questions and Falsifiable Hypotheses

RQ1. Does adaptive structural attention improve realistic vulnerability detection over the identical sequence-only CodeBERT backbone?

RQ2. Does adaptive gating outperform fixed AST, fixed DFG, and fixed AST+DFG structural biases when every model uses the same pretrained backbone and training protocol?

RQ3. Are AST and DFG signals complementary, and do learned gates reveal different usage patterns across transformer layers, heads, and vulnerability categories?

RQ4. Does structural guidance improve data efficiency, project/dataset generalization, or vulnerable-versus-patched discrimination?

RQ5. What computational overhead is introduced by structural extraction and adaptive attention, and is any performance gain still practical at inference time?

Primary hypothesis H1: ASTGuard will reduce false-positive rate at a comparable recall and/or improve PR-AUC relative to the sequence-only CodeBERT baseline. H2: adaptive gating will outperform a fixed AST+DFG bias, demonstrating that the benefit comes from selective structural use rather than simply adding more features. H3: AST and DFG ablations will show non-identical contributions, while their combination will be most useful on samples requiring both syntactic and data-dependency reasoning. H4: modest structural edge dropout during training will improve robustness to parser/alignment noise. These are hypotheses, not assumptions; negative results remain scientifically informative.

4. Data and Evaluation Setting

4.1 Primary dataset: PrimeVul

PrimeVul is the primary benchmark because it was designed to address weaknesses in earlier vulnerability datasets. Its public release contains roughly 7,000 vulnerable and 229,000 benign C/C++ functions across more than 140 CWE categories, together with de-duplication and chronological splitting intended to reduce contamination [3]. It also provides vulnerable/patched pairs and a Vulnerability Detection Score (VD-Score), which are directly aligned with ASTGuard's goal of distinguishing subtle structural security changes without overwhelming developers with false positives.

The official PrimeVul split is preserved by default. The implementation must not create a random split that mixes future examples into training. All threshold tuning occurs on validation data only. Duplicate and near-duplicate audits are performed using normalized function hashes, with a report of any overlap found across train/validation/test.

4.2 External generalization: DiverseVul

DiverseVul is used as the principal external dataset because it contains a large, diverse collection of C/C++ functions mined from vulnerability-fixing commits. The published dataset reports 18,945 vulnerable and 330,492 non-vulnerable functions covering 150 CWEs [4]. Because its labeling procedure and split assumptions differ from PrimeVul, cross-dataset evaluation is reported separately rather than mixed into the main PrimeVul score.

4.3 Supplementary benchmark: Devign

Devign is retained as a smaller, historically important benchmark for compatibility with prior graph-based vulnerability work [5]. It is not the main evidence for the paper because older benchmark construction can make comparison less realistic than PrimeVul. It is used only when the task definition and labels can be mapped fairly.

4.4 Preprocessing pipeline

C/C++ function	CodeBERT tokens	Tree-sitter AST	DFG extraction	Token-structure alignment	ASTGuard attention	Vulnerability probability
-------------------	--------------------	--------------------	-------------------	------------------------------	-----------------------	------------------------------

Tree-sitter generates AST nodes and byte spans. DFG extraction is deterministic static analysis, preferably adapted from a public, licensable implementation such as the GraphCodeBERT/CodeXGLUE data-flow extraction logic. A dedicated alignment layer maps parser byte spans and identifiers to CodeBERT subword tokens. Structural relations are stored sparsely as edge lists and converted to batched relation tensors at runtime. Any parse, alignment, or truncation failure is logged rather than silently discarded.

5. Proposed Method: ASTGuard

5.1 Base transformer

The controlled experiments use microsoft/codebert-base as the common pretrained backbone. CodeBERT uses a RoBERTa-style transformer and provides a strong code-aware sequence representation [1]. The sequence-only baseline and every ASTGuard ablation start from the same pretrained checkpoint so that differences can be attributed to the structural mechanism rather than model size or pretraining source.

5.2 Structural relations

For a tokenized function of length L , ASTGuard constructs relation masks M^{AST} and M^{DFG} in $\{0,1\}^{L \times L}$. M^{AST} links subword tokens aligned to structurally related AST nodes (initially direct parent-child relations; optional typed extensions can be added after the core study). M^{DFG} links tokens participating in definition-use and value-dependency relations. Special tokens and padding never receive structural edges. All relation tensors respect the exact truncation applied to the token sequence.

5.3 Adaptive relation-gated attention

For layer l , attention head h , query token i , and key token j , the sequence-only baseline uses the standard scaled dot-product attention score:

$$S_{seq,ij}(l,h) = Q_i(l,h) (K_j(l,h))^T / \sqrt{d_h}$$

The attention probabilities and head output are then:

$$A_{ij}^{(l,h)} = \text{softmax}_j(S_{ij}^{(l,h)})$$

$$O_i^{(l,h)} = \sum_j A_{ij}^{(l,h)} V_j^{(l,h)}$$

These equations are retained by every controlled variant. ASTGuard changes only the score S before softmax; it does not replace the transformer attention operation.

For each structural relation r in $\{\text{AST}, \text{DFG}\}$, define a binary token-relation matrix:

$$M_{ij}^r = 1 \text{ if tokens } i \text{ and } j \text{ are connected by relation } r; \text{ otherwise } M_{ij}^r = 0.$$

The fixed-structure baseline adds learned relation-specific structural magnitudes without contextual gates:

$$S_{\text{fixed},ij}^{(l,h)} = Q_i K_j^T / \sqrt{d_h} + \sum_{r \in \{\text{AST}, \text{DFG}\}} \beta_r^{(l,h)} M_{ij}^r$$

AST-only and DFG-only baselines are obtained by enabling only the corresponding relation. The AST+DFG-fixed baseline enables both relations. This creates the controlled progression Sequence-only $\rightarrow$ Fixed Structure $\rightarrow$ Adaptive Structure.

ASTGuard introduces a contextual gate for each query token, attention head, layer, and relation:

$$g_{i,r}^{(l,h)} = \text{sigmoid}(W_r^{(l,h)} h_i^{(l-1)} + b_r^{(l,h)})$$

The proposed adaptive structural attention score is:

$$S_{\text{ASTGuard},ij}^{(l,h)} = Q_i K_j^T / \sqrt{d_h} + \sum_{r \in \{\text{AST}, \text{DFG}\}} g_{i,r}^{(l,h)} \beta_r^{(l,h)} M_{ij}^r$$

where $\beta_r^{(l,h)}$ is a learned relation magnitude and $0 \leq g_{i,r}^{(l,h)} \leq 1$. The model can therefore suppress a structural signal for one token while emphasizing it for another. The structural term is a soft additive prior: ordinary semantic token-to-token attention remains unrestricted.

For binary vulnerability prediction, the default training objective is weighted binary cross-entropy to account for class imbalance:

$$L_{BCE} = -(1/N) \sum_n [w_1 y_n \log(p_n) + w_0 (1-y_n) \log(1-p_n)]$$

Class weights are estimated only from the training split. Threshold selection, early stopping, and model selection use validation data; test labels are never used for tuning.

A required implementation invariant is that when $\text{beta_AST} = \text{beta_DFG} = 0$ (or all structural contributions are otherwise disabled), the ASTGuard forward pass must numerically match the sequence-only CodeBERT forward pass within floating-point tolerance.

5.4 Structural dropout and interpretability

A robustness variant randomly removes a configurable fraction of AST/DFG edges during training only. This tests whether the model learns a brittle dependence on exact parser output. During evaluation, gate values are aggregated by relation, layer, head, label, CWE (where available), and correctness. This analysis is central to the research contribution because it can show not only whether ASTGuard performs better, but how it chooses between syntax and data flow.

6. Baselines and Ablation Plan

The baseline strategy has two purposes. Controlled baselines isolate the effect of adaptive gating under an identical backbone. External baselines place the result in the broader vulnerability-detection literature. The paper's strongest causal comparison is ASTGuard versus the same CodeBERT with no structure and with fixed structure; cross-architecture comparisons are supporting evidence, not substitutes for the controlled study.

6.1 Mandatory controlled baselines (same backbone)

ID	Model	Backbone	Structure	Why it matters
B0	CodeBERT sequence-only	CodeBERT	None	Primary control. Measures the value of any structural mechanism.
B1	AST-fixed	CodeBERT	AST soft bias; no contextual gate	Tests AST information without adaptivity.
B2	DFG-fixed	CodeBERT	DFG soft bias; no contextual gate	Tests data-flow information without adaptivity.
B3	AST+DFG-fixed	CodeBERT	Both relation biases; no contextual gate	Directly tests whether adaptive gating beats simply adding both structures.
M	ASTGuard	CodeBERT	Adaptive AST+DFG gates	Proposed method.
M+	ASTGuard + structural dropout	CodeBERT	Adaptive AST+DFG + edge dropout	Robustness extension; secondary proposed variant.

6.2 Strong external baselines

Baseline	Description	Priority	Role in study
GraphCodeBERT	Transformer with data-flow-aware pretraining and graph-guided masked attention [2].	Mandatory	Closest established structure-aware transformer comparison.
LineVul	CodeBERT-based transformer for vulnerability prediction and line-level localization [7].	Mandatory	Strong vulnerability-specific transformer baseline using similar pretrained family.
ReGVD	Graph neural network built from token graphs initialized with pretrained code-model embeddings [6].	Preferred	Represents a modern graph-centric vulnerability detector.
Devign	Graph neural network using rich program semantics for function-level vulnerability classification [5].	Preferred / legacy	Historically important graph-based comparison and Devign benchmark reference.
UniXcoder classifier	Sequence-only pretrained code model; no ASTGuard mechanism.	Optional	Checks whether gains persist against a different strong pretrained code encoder; not used for causal claims.

Implementation priority is deliberate. CodeBERT, the three fixed-structure ablations, GraphCodeBERT, and LineVul are mandatory for the main report. ReGVD and Devign are included when their public reproduction pipelines can be run fairly on the chosen split without consuming the entire project schedule. If a graph baseline cannot be reproduced reliably, the limitation is reported instead of substituting unverifiable numbers from a different dataset or split.

6.3 Sanity baselines

A trivial class-prior predictor and, optionally, a simple bag-of-tokens logistic regression are included only as sanity checks. They are not presented as competitive baselines. Their purpose is to detect pipeline mistakes, confirm metric behavior under imbalance, and show that the learned models are doing more than exploiting label frequency.

7. Experimental Design

Experiment	Data	Models	Replicates	Purpose
E1 Main comparison	PrimeVul	B0-B3, M, M+, GraphCodeBERT, LineVul	3 seeds	Core evidence for RQ1/RQ2.
E2 Structure ablation	PrimeVul	B1, B2, B3, M	3 seeds	AST vs DFG vs combined; fixed vs adaptive.
E3 Layer placement	PrimeVul	ASTGuard with early/middle/late/all selected layers	≥ 1 seed screening; 3 for finalists	Where structural influence is most useful.
E4 Data efficiency	PrimeVul	CodeBERT vs ASTGuard	10/25/50/100% train data	Does structure help with fewer labels?
E5 Cross-dataset	Train PrimeVul $\rightarrow$ test DiverseVul	CodeBERT, GraphCodeBERT, ASTGuard	3 seeds if feasible	Generalization beyond the source distribution.
E6 Paired evaluation	PrimeVul vulnerable/patched pairs	CodeBERT, GraphCodeBERT, ASTGuard	Same trained checkpoints	Can the model distinguish a vulnerability from its fix?
E7 Structural robustness	PrimeVul	ASTGuard with 0/5/10/20% edge corruption/dropout	≥ 1 seed screening	Sensitivity to imperfect parsing/relations.
E8 Efficiency	PrimeVul test	CodeBERT, B3, ASTGuard	Fixed hardware	Latency, GPU memory, parameters, preprocessing cost.
E9 Gate analysis	PrimeVul test	ASTGuard	Final checkpoints	Relation/layer/head usage and qualitative cases.

All hyperparameters are selected without inspecting the test labels. The validation set is used for early stopping, threshold selection, and structural-bias/gate hyperparameters. When screening many architectural settings, only a small number of finalists are run across all seeds to control compute and reduce multiple-comparison overfitting.

8. Training Protocol and Implementation Details

- Optimizer: AdamW with a small pretrained-model learning rate (initial search around $1e-5$ to $5e-5$), linear warmup, weight decay, gradient clipping, and early stopping on validation PR-AUC or F1 depending on the registered primary criterion.
- Sequence length: up to 512 subword tokens for the initial CodeBERT study. Truncation is applied consistently to tokens and structural edges. A length-stratified error analysis checks whether truncation disproportionately harms vulnerable functions.
- Class imbalance: weighted binary cross-entropy or focal loss may be evaluated on validation data. The natural test distribution is never resampled. Thresholds are tuned only on validation data.
- Reproducibility: at least three random seeds for main results, deterministic seeds where supported, exact configuration capture, environment/CUDA metadata, git commit hash, and saved predictions for every run.
- Efficiency: mixed precision, gradient accumulation, gradient checkpointing, and sparse structural storage are supported. Dense relation masks are created only per batch when required by the attention implementation.
- No proprietary APIs: training and preprocessing run locally with PyTorch, Hugging Face Transformers, Tree-sitter, and public dataset/model downloads.

9. Metrics, Statistical Analysis, and Decision Criteria

Because PrimeVul is highly imbalanced, accuracy is reported but not treated as the main success metric. The primary evaluation emphasizes PR-AUC, precision, recall, F1, MCC, FPR, FNR, confusion matrices, and PrimeVul's VD-Score where the official implementation is compatible. ROC-AUC is included for completeness but interpreted cautiously under strong imbalance.

Two operating-point analyses are especially important: (1) FPR at a fixed or matched recall, and (2) recall at a constrained false-positive rate. These directly test the stated deployment motivation. On PrimeVul paired samples, the study reports whether a model correctly assigns higher vulnerability probability to the vulnerable member than to its patched counterpart, along with pair-level correctness where supported by official identifiers.

Main metrics are summarized across seeds with mean and standard deviation. Bootstrap confidence intervals are computed on final test predictions when practical. The paper will not treat a single-seed increase as evidence of improvement. If ASTGuard gains F1 but substantially increases FPR or compute, that tradeoff is reported rather than hidden.

10. Model Analysis

ASTGuard exposes interpretable gate variables that can be inspected without claiming that attention is a complete explanation of model reasoning. The analysis will measure mean/median gate values, variance, fraction near zero, and fraction near one for AST and DFG relations at each layer/head. These values will be compared across vulnerable and benign samples, correct and incorrect predictions, and CWE categories with sufficient support.

Qualitative case studies will focus on matched vulnerable/patched pairs and on examples where the sequence baseline fails but ASTGuard succeeds, where ASTGuard fails but the baseline succeeds, and where both fail. Visualizations will highlight the source tokens, structural edges active for selected attention heads, and gate magnitudes. The objective is to identify recurring patterns rather than cherry-pick isolated examples.

11. Data Access, API Keys, and Reproducibility

The planned core pipeline requires no paid or proprietary API keys. PrimeVul and DiverseVul are publicly released datasets; Devign/CodeXGLUE resources and the cited baseline implementations are public research artifacts. CodeBERT and GraphCodeBERT are public pretrained models. Tree-sitter parsing and DFG extraction run locally. A Hugging Face token may be supported as an optional convenience for rate-limited or authenticated downloads, but it is not a scientific dependency of the method. Public GitHub cloning does not require a GitHub API key; public Google Drive downloads normally do not require a developer key. If an upstream host later changes its access policy, the code must fail with an actionable message rather than silently replacing the dataset.

The repository will therefore be reproducible from public artifacts: download -> verify checksums/metadata -> preprocess -> train -> evaluate -> aggregate seeds -> generate tables/figures. Raw datasets, large caches, and checkpoints remain gitignored. Every reported table cell must map to an actual run directory and stored prediction file.

12. Key Risks and Mitigations

Risk	Why it matters	Mitigation
Novelty overlap	A closely related adaptive structural-attention mechanism may already exist.	Perform a targeted literature review before making first-of-kind claims; position contribution as an evaluated mechanism if overlap is found.
DFG extraction errors	Static analysis can fail on incomplete functions/macros or produce noisy edges.	Log failures, measure parse coverage, use structural dropout, and run robustness/corruption tests.
Token-node alignment	Tree-sitter byte spans and BPE subwords can mismatch.	Build explicit offsets, unit tests, visualization checks, and round-trip alignment assertions.
Class imbalance	A high accuracy model may simply predict benign.	Use PR-AUC, FPR, MCC, VD-Score, constrained operating points, and validation-only threshold tuning.
Compute burden	External graph baselines and multiple seeds can be expensive.	Prioritize controlled baselines; screen configurations with one seed; run three seeds only for final models.
Dataset mismatch	Cross-dataset labels/splits may not be semantically identical.	Document mapping assumptions and omit invalid comparisons rather than forcing them.
Negative result	Adaptive gating may not improve performance.	Use ablations, robustness, paired analysis, and gate behavior to determine why; report the null result faithfully.

13. Deliverables and Success Criteria

- Reproducible dataset adapters for PrimeVul, DiverseVul, and optionally Devign, including split preservation and leakage reports.
- Deterministic Tree-sitter AST extraction, DFG extraction, and token-structure alignment with explicit failure logs.
- CodeBERT baseline, fixed AST/DFG baselines, adaptive ASTGuard, and structural-dropout variant.
- External baseline runners for GraphCodeBERT and LineVul; ReGVD/Devign adapters when reproducible within schedule.
- Main-result, ablation, data-efficiency, cross-dataset, paired, efficiency, robustness, and gate-analysis artifacts.
- A results manifest linking every paper table/figure to run IDs, configurations, seeds, predictions, and checkpoints.
- Final research report describing positive or negative findings without fabricating or selectively omitting results.

Primary success criterion: compared with the same sequence-only CodeBERT, ASTGuard should show a statistically credible improvement in the precision/false-positive/PR-AUC tradeoff while keeping recall and computational overhead reasonable. A secondary success criterion is that adaptive AST+DFG gating should outperform the fixed AST+DFG mechanism, supporting the specific novelty claim. If that second criterion fails, the paper must not attribute improvements to adaptivity.

14. Four-Month Work Plan

Period	Milestones
Month 1	Literature audit; public dataset acquisition; leakage audit; Tree-sitter AST; DFG extraction; token alignment; CodeBERT baseline; smoke tests.
Month 2	Fixed AST/DFG attention; adaptive gating; structural dropout; unit/integration tests; GraphCodeBERT and LineVul baseline integration.

Period	Milestones
Month 3	PrimeVul 3-seed main experiments; ablations; data-efficiency; paired evaluation; external/cross-dataset runs; efficiency measurement.
Month 4	ReGVD/Devign if feasible; gate analysis; robustness tests; confidence intervals; figures/tables; error analysis; report and reproducibility audit.

15. Expected Paper Story

The intended paper is not "we added AST and DFG to CodeBERT." The paper asks a more specific question: should explicit program structure influence every token and attention head equally? ASTGuard operationalizes the alternative by learning token-conditioned relation gates inside self-attention. The main evidence then follows a clean progression: sequence-only CodeBERT -> fixed AST/DFG structure -> adaptive ASTGuard -> external structure-aware and vulnerability-specific baselines. If adaptive gating improves realistic PrimeVul behavior, the paper can argue that selective structural priors are useful. If fixed structure performs equally well, the result shows that adaptivity is unnecessary. If all structural variants fail, the result challenges the assumption that explicit AST/DFG bias is beneficial once strong pretrained code representations and realistic benchmark controls are used. Each outcome answers a meaningful research question.

References

- [1] Feng, Z. et al. "CodeBERT: A Pre-Trained Model for Programming and Natural Languages." arXiv:2002.08155, 2020. <https://arxiv.org/abs/2002.08155>
- [2] Guo, D. et al. "GraphCodeBERT: Pre-training Code Representations with Data Flow." arXiv:2009.08366, 2020. <https://arxiv.org/abs/2009.08366>
- [3] Ding, Y. et al. "Vulnerability Detection with Code Language Models: How Far Are We?" arXiv:2403.18624; accepted at ICSE 2025. <https://arxiv.org/abs/2403.18624>
- [4] Chen, Y., Ding, Z., Alowain, L., Chen, X., and Wagner, D. "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection." RAID 2023; arXiv:2304.00409. <https://arxiv.org/abs/2304.00409>
- [5] Zhou, Y. et al. "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks." arXiv:1909.03496, 2019. <https://arxiv.org/abs/1909.03496>
- [6] Nguyen, V.-A. et al. "ReGVD: Revisiting Graph Neural Networks for Vulnerability Detection." arXiv:2110.07317, 2021. <https://arxiv.org/abs/2110.07317>
- [7] Fu, M. and Tantithamthavorn, C. "LineVul: A Transformer-based Line-Level Vulnerability Prediction." MSR 2022. Replication package: <https://github.com/aws-sm-research/LineVul>
- [8] DLVulDet. "PrimeVul: Vulnerability Detection Dataset." Public repository and VD-Score implementation. <https://github.com/DLVulDet/PrimeVul>
- [9] Wagner Research Group. "DiverseVul" public dataset repository. <https://github.com/wagner-group/diversevul>

Appendix A. Minimum Experiment Set for a Defensible Final Report

If time or compute becomes constrained, the following set should be protected before optional experiments. This prevents the project from becoming broad but scientifically inconclusive.

Priority	Items	Requirement
Must run	CodeBERT sequence-only, AST-fixed, DFG-fixed, AST+DFG-fixed, ASTGuard	Same PrimeVul split, same backbone, same training protocol, 3 seeds for sequence-only/fixed-combined/ASTGuard.
Must run external	GraphCodeBERT, LineVul	Retrained/evaluated on the same PrimeVul setting whenever their implementation supports it.
Must analyze	PR-AUC, precision, recall, F1, MCC, FPR/FNR, VD-Score, vulnerable/patched pairs	These support the deployment and realistic-benchmark motivation.

Priority	Items	Requirement
Must validate	Zero-structural-bias equivalence test; split/leakage audit; threshold only on validation	Protects causal validity and prevents accidental test leakage.
Preferred	ReGVD and Devign; cross-dataset PrimeVul -> DiverseVul; structural robustness	Adds broader context and generalization evidence.
Optional	UniXcoder classifier; typed AST relations; additional backbones	Useful extensions only after the core study is complete.